Week 7: Running what you build

AI Development Course

Tested with Docker Engine 29.8.0 on Linux (WSL 2, Intel/AMD chip), not Docker Desktop, and the official python:3.12-slim image (Python 3.12.15), 12 October 2026. Also tested signed in on Windows (WSL 2) with Docker Desktop 4.97.0, 29 October 2026. Installing Docker Desktop, the Mac, Windows Home and GitHub are being tested.

Today

- Why containers: "works on my machine"
- Image vs container
- Install Docker Desktop (**before class**)
- Branch, then Claude adapts the deal screener's `Dockerfile` and `.dockerignore`
- Build, run your program and your tests in a container
- Check the files with your week 6 `safe-and-private` reviewer
- Push, read Files changed, merge
- The assignment, and next week

"It works on my machine"

You send your project to a classmate. It fails on their laptop:

- a different version of Python, or no Python at all
- a package your project needs is missing

Nothing is wrong with your code. Everything **around** it is different.

A **container** packs your program with everything around it: the language (Python 3.12), the packages, your files. Docker runs it the same way on a Mac with an Apple chip, an Intel Mac and Windows. Inside, it's always a small Linux computer.

Image vs container: a recipe and a dish

Docker word	In the kitchen	What it is
`Dockerfile`	The recipe card you write	Start from this base, add these files, run this
Image	The recipe, every ingredient measured and packed	What `docker build` makes. It doesn't change
Container	A dish cooked from the recipe	What `docker run` starts. Cook many, each the same; `--rm` clears the plate
`.dockerignore`	What stays out of the kitchen	Files Docker must **not** pack: secrets, git history, add-on folders

Build the image once. Run a container whenever you use the program.

The deal screener's `Dockerfile`

```
# Start from the official Python image. 3.12 keeps Python on version 3.12 (it still
# gets small fixes); never use "latest", which can change under you.
FROM python:3.12-slim

# Work in a folder called /app inside the container.
WORKDIR /app

# Copy this folder into /app, leaving out everything listed in .dockerignore.
COPY . .

# Run as "nobody", a user with no special powers (optional: if your program writes
# files inside /app and gets "Permission denied", remove this line or ask Claude).
USER nobody

# The command that runs when the container starts: the deal screener.
CMD ["python", "-m", "dealscreener"]
```

example-project/Dockerfile, comments shortened. No install line: it needs no packages. "Pinned" is the course's word for a fixed version like 3.12; Docker's page doesn't use it.

The deal screener's `.dockerignore`

What Docker leaves **out** of the image (shortened):

```
.git
**/.venv
**/venv
**/node_modules
**/__pycache__
**/.env
**/.env.*
**/*.pem
**/*.key
.claude
docs
REVIEW.md
```

- `**/` means "in this folder or any folder below it". A bare name like `.venv` only matches at the top
- Also logs (`*.log`), spreadsheet exports (`*.csv`), `.superpowers` , `*.pyc`

Full file with a reason for each group: example-project/.dockerignore. Docker's docs: docs.docker.com/build/concepts/context/

Install Docker Desktop: before class

- **Licence:** free for "Personal use", "Education", small businesses (fewer than 250 employees AND less than $10 million revenue) and non-commercial open source. Employer laptop at a bigger company? Ask them first
- **Admin rights:** Mac, "you aren't asked for a password"; Windows, per-user install "needs no administrator privileges". School or work laptop that blocks installs? Ask before class
- **Disk:** Docker gives no number. The deal screener's image: about 180 MB
- `docker` only works while **Docker Desktop is running**

docs.docker.com/subscription/desktop-license/ and the install pages. Screens and prompts are being tested.

Install on a Mac (being tested)

Apple menu > **About This Mac**: **Chip** `Apple M1` (Apple chip) or **Processor** Intel (Intel Mac).

Requirements: "the current and two previous major macOS releases", "At least 4 GB of RAM".

1. docs.docker.com/desktop/setup/install/mac-install/: download **Docker Desktop for Mac with Apple silicon** or **... with Intel chip**
2. Double-click `Docker.dmg`, drag Docker to **Applications**
3. Double-click `Docker.app`; accept the **Subscription Service Agreement**
4. Apple chip: Rosetta 2 recommended, "no longer strictly required"
5. macOS asks to allow something for Docker? Allow it (being tested)

Install on Windows (WSL 2, being tested)

1. `winver`: Windows 10 22H2 (build 19045), or Windows 11 23H2 (build 22631) or higher. Edition (Settings > System > About): Docker lists Pro, Enterprise, Education. **Home: ask your instructor first** (being tested)
2. In PowerShell: `wsl --version` (2.1.5 or later; `wsl --update` if older), `wsl.exe -l -v` (Ubuntu VERSION 2)
3. Run `Docker Desktop Installer.exe`: **per-user**; tick **Use WSL 2 instead of Hyper-V**; **Close**
4. Start: search **Docker**, choose **Docker Desktop**; **Accept** the agreement
5. **Settings > Resources > WSL Integration**: Ubuntu on (on by default), **Apply**

From now on, every `docker` command goes in the **Ubuntu window**, never PowerShell.

Also: 8GB RAM, virtualization on in BIOS/UEFI. docs.docker.com/desktop/setup/install/windows-install/ and docs.docker.com/desktop/features/wsl/

Check it works

In Terminal (Mac) or the Ubuntu window (Windows):

```
docker --version
docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
...

Hello from Docker!
This message shows that your installation appears to be working
correctly.
```

Our test: Docker version 29.8.0, build 88096ef. hello-world uses latest, fine for a one-off test; your own Dockerfile never does.

Branch first, and what to answer

`cd ~/semester-project && claude`, `manual mode on`, clean `git status`, then "Switch to main and pull ... make a new branch called **containerize** and switch to it."

Claude asks to	Choose
Edit `Dockerfile`, `.dockerignore`, `README.md`, `REVIEW.md`	**Yes**
`git add` / `commit` / `push` on `containerize`, `gh pr create`	plain **Yes**
`docker build -t semester-project .`, `docker run --rm semester-project ...`	**Yes**
`docker system prune -a`	**No**
`--privileged`, or mounting your home folder (`-v ~:...`)	**No**
`FROM ...:latest`, or no version	**No**
`docker login`, `docker push`	**No**
Change `main`, `reset --hard`, `--force`, `git clean`, `rm -rf`	**No**

Copy, then adapt

Second terminal window:

```
cd ~/ai-dev-course && git pull
cd ~/semester-project
cp ~/ai-dev-course/example-project/Dockerfile ~/ai-dev-course/example-project/.dockerignore .
```

In Claude Code (shortened; full prompt in the README, step 3c):

```
I copied Dockerfile and .dockerignore from a Python project, a rental deal
screener, as a model. ... adapt both files to my project. Rules: start from an
official image on Docker Hub for my language, pinned to a version, never latest
(if my project is Python, use python:3.12-slim, as the model does) ... Never copy
a .env file, passwords, keys or other secrets into the image ... Keep USER nobody
unless my program writes files inside the container ... Don't add --privileged.
If my program isn't typed into, tell me the exact docker run command, ask
before adding a port, and never mount anything except this project folder.
Keep a plain-English comment above every line. Don't build or run anything yet, and don't commit.
```

Read your Dockerfile

```
Explain each line of the Dockerfile in plain English.
```

Check:

- `FROM`: an **official** image with a **version**, never `latest`
- Nothing copies a `.env` file or a secret
- `CMD`: runs **your** program, not the deal screener
- A line you don't follow? Ask about it until you do

Build and run

Second terminal window, in `~/semester-project`:

```
docker build -t semester-project .
docker run -it --rm semester-project
```

- `-t semester-project`: the image's name. `.`: "this folder"
- `-it`: lets you type into the program (`-i` keeps your typing connected, `-t` acts like a terminal)
- No `-it`? The deal screener prints `Stopped before all the numbers were typed.`
- `--rm`: deletes the container when it ends. The image stays

Type your **week 5 hand-worked example**: same answer? **Screenshot.** Not typed into? Use the run command from step 4b with your example as input (a web page: a screenshot of the page in your browser).

Near the end it prints "naming to docker.io/library/semester-project:latest" (Docker Desktop adds an "unpacking to" line): that latest is your own image's label, fine.

Tests in the container, and what's inside

Ask Claude for your test command (don't run it), then type it yourself:

```
docker run --rm semester-project python -m unittest
```

```
Ran 39 tests in 0.002s

OK
```

Screenshot. Then check what went in:

```
docker run --rm semester-project ls -a
```

```
.  ..  .dockerignore  Dockerfile  PLAN.md  README.md  dealscreener  tests
```

No `.env`, `.git`, `.venv`, `.claude`, `__pycache__`. Then commit: "Run the project in Docker".

Deal screener output (its image is called dealscreener; ls -a prints one name per line). Node.js: maybe npm test (being tested).

Review it with `safe-and-private`

```
Run the safe-and-private reviewer as a subagent on Dockerfile and .dockerignore
only. Then add a new section at the end of REVIEW.md, headed "## Week 7:
Dockerfile review", with one table with the columns #, Label, Finding
(file:line), Found by, Decision, Reason. Number the findings D1, D2 and so on,
so they don't clash with week 6's numbers. Sort the rows: Must fix first, then
Should fix, then Nice to have. Keep each finding's wording close to what the
reviewer wrote, and keep its file and line numbers. Leave Decision and Reason
empty. Do not add any finding the reviewer did not report, and do not change
any file except REVIEW.md. After the table, list in a few words the areas where
the reviewer said "Nothing found".
```

Yes to `REVIEW.md` (an edit, or a `cat >> REVIEW.md` command), **No** to any other file. Commit: "Review of the Dockerfile".

Triage and fix, as in week 6

The deal screener's reviewer: 4 findings, all Nice to have. 2 `Fix`, 2 `Won't fix`.

#	Finding	Decision	Reason
D2	`.venv` and `venv` match only at the top folder	Fix	A bare name only matches at the top, so now `**/`
D3	`FROM` uses a tag, not a digest, so the image can change	Won't fix	A digest stops security fixes arriving; `3.12-slim` is the course's pin

- Check each finding first; every `Won't fix` needs a reason
- "Fix finding D1 in REVIEW.md. Change only Dockerfile or .dockerignore ..."
- After each fix: build again, run the tests, `ls -a` again

Shortened. Full section: example-project/REVIEW.md, "Week 7: Dockerfile review".

Finish

1. "Add a short section called 'How to run with Docker' to README.md ..." (build, test, run commands)
2. "Push the containerize branch to GitHub and open a pull request into main titled 'Run in Docker'."
3. **Files changed:** `Dockerfile`, `.dockerignore`, `README.md`, `REVIEW.md` (and any fix): nothing else. **No** `.env`
4. **Merge pull request**, **Confirm merge** (or **Squash and merge**; being tested), then "Switch back to main and pull the latest changes from GitHub."

Tidy up, optional, typed yourself: `docker image prune` deletes only unused leftovers (`[y/N]`). Never `docker system prune -a`.

Committed to main? Week 5's rescue steps, with rescue-containerize.

When it goes wrong

What you see	Most likely
`Cannot connect to the Docker daemon` (being tested)	Docker Desktop isn't running: start it, wait until it says it is running, try again
`docker` not found, or Ubuntu suggests `snap install docker` (being tested)	Don't install Docker in Ubuntu: **WSL Integration**: Ubuntu on, **Apply**, new Ubuntu window
Line starts `PS C:\`	You're in PowerShell: use the Ubuntu window
`Unexpected WSL error`	Virtualization off in BIOS/UEFI: office hours
`Stopped before all the numbers were typed.`	You left out `-it`
`Permission denied` or `EACCES` (program or tests)	Remove `USER nobody`, or ask Claude
`exec format error`, `no matching manifest` (being tested)	Made for another chip: ask Claude for official images for your chip
`ls -a` shows `.env` or `.git`	Add it to `.dockerignore` with `**/`, build, check again

Assignment

- The **merged PR** address: `Dockerfile`, `.dockerignore`, README section, `REVIEW.md`
- Screenshot: **tests passing in the container**
- Screenshot: **your program in the container** on your own example, hand-worked answer beside it
- Reflection: one thing the container made you notice; what `safe-and-private` said and what you did

Criterion	Points
Dockerfile and `.dockerignore` merged by PR: pinned official base, last line runs your program, nothing secret or unneeded copied in	4
Tests and program in the container, screenshots, hand-worked comparison	3
Reflection	3

Next week: Debugging with AI, and midterm check-in. Bring your merged containerize PR.